UI Guidelines

Version: 1.0

Purpose

This document defines the user interface and interaction guidelines for the Trading Cockpit.
The UI shall support professional trading workflows through:
• clarity
• operational transparency
• efficient interaction
• high information density
• preserved user context
• predictable behaviour
The Trading Cockpit is a professional desktop workspace.
The UI shall prioritize trading workflow efficiency over decorative presentation.

UI Design Principles

The Trading Cockpit follows these principles:
• Consistency
• Clarity
• Efficiency
• Transparency
• Predictability
• Responsiveness
• Accessibility
• Context Preservation
Every UI element shall support a defined user workflow.
Avoid visual or interactive complexity without measurable user value.

Trading Workflow Orientation

The UI shall support the primary trading workflow:
1. Observe the market.
2. Identify relevant instruments.
3. Evaluate trading candidates.
4. Review market, portfolio and risk context.
5. Make a trading decision.
6. Prepare and validate an order.
7. Execute and monitor the order.
8. Monitor the resulting position.
9. Review the trading outcome.
Navigation and workspace behaviour shall minimize unnecessary context changes during this workflow.

Main Application Layout

The Trading Cockpit uses a configurable dockable workspace.
Primary application areas:
• Application Navigation
• Workspace
• Status Bar
• Notification Area
• Command Palette

The workspace contains user-configurable widgets.
The application shall not enforce one fixed trading layout.

Workspace Design

The workspace is a core product capability.
Users shall be able to:
• dock widgets
• resize widgets
• move widgets
• hide widgets
• restore widgets
• arrange workflow-specific layouts
• preserve workspace state
• restore the previous working context
Workspace interaction shall remain predictable.
Invalid workspace state shall not prevent application startup.

Widget Design

Every widget shall:
• have a clear title
• have one primary responsibility
• identify its active context where relevant
• support resizing
• support docking
• support theming
• expose loading state
• expose error state
• expose unavailable state where applicable
• expose stale data state where applicable
Widgets shall remain visually consistent.
Widget controls shall not unnecessarily consume space required for trading information.

Widget Header

Widget headers should provide only relevant controls.
Typical header elements may include:
• widget title
• active instrument
• data state
• refresh state
• widget actions
• context actions
Avoid permanently displaying rarely used actions.
Secondary actions should use:
• context menus
• overflow menus
• Command Palette commands

Shared Instrument Context

Compatible widgets may participate in shared instrument context.
Example:
Watchlist → Price Chart → Decision Center → Order Entry

When a user selects an instrument in a publishing widget, compatible following widgets may update automatically.
Context-aware widgets shall clearly display the active instrument.
Users shall be able to understand why a widget changed context.
Unexpected hidden context changes are not allowed.
Widgets may:
• publish context
• follow context
• publish and follow context
• remain context-independent
Context behaviour shall follow the Widget Catalog.

Context Preservation

The UI shall preserve user context whenever practical.
Examples:
• selected instrument
• selected candidate
• active workspace
• widget layout
• filters
• sorting
• selected timeframe
Opening another widget shall not unnecessarily reset existing workflow context.
Application restart should restore the previous workspace where technically and operationally safe.

Data State Representation

Trading information shall expose its operational state.
Standard data states:
• Loading
• Ready
• Stale
• Unavailable
• Disconnected
• Error
The UI shall never silently display stale data as current data.
Data state indicators shall remain visually consistent across widgets.
Where relevant, display:
• source
• timestamp
• last update
• connection state

Loading State

Loading states shall communicate that data is being retrieved or processed.
Use:
• compact progress indicators
• skeleton states where appropriate
• status text for longer operations
Avoid blocking the complete application for widget-specific loading operations.

Stale Data State

Stale trading data is operationally relevant.
Widgets displaying stale data shall:
• visibly indicate stale state
• preserve the last available value where useful
• expose the last update timestamp where relevant
Stale data shall not visually appear identical to current data.

Unavailable State

Unavailable information shall be represented explicitly.
Examples:
• `Unavailable`
• `N/A`
• `No data`
Do not replace unavailable financial information with:
• zero
• estimated values
• previous values without stale indication
The UI shall not invent missing financial information.

Disconnected State

External connection failures shall be visible.
Examples:
• broker disconnected
• market data disconnected
• external service unavailable
The UI shall communicate:
• affected capability
• current connection state
• operational impact
Disconnected state shall not be hidden only in logs.

Error State

User-facing errors shall explain:
• what failed
• the operational impact
• whether user action is required
Avoid exposing internal stack traces in normal UI views.
Detailed technical information belongs in structured logs.

Information Density

The Trading Cockpit is a professional information-dense application.
The UI should:
• maximize useful trading information
• minimize decorative whitespace
• preserve visual hierarchy
• avoid unnecessary large controls
• support compact tables
• support efficient scanning
High information density shall not reduce readability.

Tables

Tables are primary Trading Cockpit components.
Tables shall support where appropriate:
• column sorting
• filtering
• column resizing
• column reordering
• keyboard navigation
• row selection
• multi-selection
• context menus
• persistent column state
Numeric values should be aligned consistently.
Financial values shall use consistent formatting.

Financial Value Formatting

Financial values shall use explicit formatting rules.
Examples:
• prices
• quantities
• percentages
• currency values
• P&L; values
• exposure values
Formatting shall consider:
• instrument precision
• currency
• sign
• unavailable state
Positive and negative values may use semantic visual indicators.
Colour shall not be the only indicator of value meaning.

Time and Timestamp Display

Trading information frequently depends on time.
Timestamps shall:
• use a defined timezone
• remain consistent within a view
• expose timezone context where ambiguity exists
The application shall distinguish between:
• market timestamp
• broker timestamp
• local application timestamp
Do not silently mix timezone contexts.

Navigation

Navigation shall minimize workflow interruption.
Primary navigation should provide access to major product capabilities.
Use:
• keyboard shortcuts
• Command Palette
• context actions

• direct widget interaction
Avoid deep menu hierarchies.
Navigation shall preserve workspace context.

Command Palette

The Command Palette provides keyboard-driven application control.
Commands may include:
• open widget
• close widget
• switch workspace
• select instrument
• execute application command
• open settings
• trigger refresh
Commands shall respect current application state.
Unavailable commands shall be disabled or omitted.

Keyboard Interaction

Professional workflows shall support efficient keyboard interaction.
Where practical:
• tables support keyboard navigation
• dialogs support Enter and Escape
• common actions provide shortcuts
• focus movement remains predictable
• Command Palette is keyboard accessible
Keyboard shortcuts shall be documented and consistent.

Mouse Interaction

The UI shall support:
• single selection
• multi-selection where appropriate
• drag and drop where useful
• context menus
• docking
• resizing
Avoid hidden mouse-only actions for critical workflows.

Notifications

Notifications shall communicate operationally relevant information.
Notification levels:
• Information
• Warning
• Error
• Critical
Examples:
• broker disconnected
• market data stale
• order rejected
• reconciliation discrepancy
• background service failure
Notifications shall not be used for routine noise.

Critical trading events shall remain visible until appropriately handled.

Order Workflow UI

Order workflows are business-critical.
Order Entry shall clearly display:
• instrument
• action
• quantity
• order type
• price parameters
• validation state
Before submission, the user shall be able to review relevant order parameters.
Invalid orders shall not be submitted.
Broker acknowledgement, rejection and execution state shall be visible.
The UI shall not imply successful submission before broker acknowledgement is available.

Destructive and Critical Actions

Critical actions require explicit interaction.
Examples:
• order submission
• order cancellation
• position close
• configuration reset
• workspace reset
Confirmation requirements shall depend on operational risk.
Avoid confirmation dialogs for routine low-risk actions.
Critical actions shall not be triggered accidentally through ambiguous controls.

Portfolio and Risk UI

Portfolio and risk information shall clearly identify:
• current state
• unavailable state
• stale state
• source where relevant
• timestamp where relevant
Local and broker-derived state shall remain distinguishable where operationally important.
Reconciliation discrepancies shall be visible.

Visual Design

The Trading Cockpit shall use:
• consistent spacing
• consistent typography
• consistent component sizing
• semantic visual indicators
• restrained visual hierarchy
• professional information density
Avoid excessive:
• gradients
• animations
• decorative shadows
• oversized typography

• visual effects without functional value

Colour Usage

Colour shall communicate meaning.
Typical semantic uses:
• positive
• negative
• warning
• error
• disconnected
• stale
• selected
• active
Colour shall remain consistent across widgets.
Colour shall not be the only method of communicating critical state.

Theme Support

The Trading Cockpit shall support:
• Dark Theme
• Light Theme
The initial implementation may prioritize Dark Theme.
Widgets shall use shared theme definitions.
Hard-coded widget-specific colours should be avoided.

Responsiveness

The UI shall remain responsive during normal operation.
Rules:
• never block the UI thread with external integrations
• avoid uncontrolled refresh rates
• update only affected UI components where practical
• isolate expensive processing
• provide progress feedback for long operations
UI responsiveness is a product requirement.

Accessibility

The application should support:
• keyboard-only operation
• scalable fonts
• sufficient contrast
• non-colour state indicators
• predictable focus behaviour
• screen reader compatibility where practical
Accessibility requirements shall be considered during widget design.

UI State Persistence

Persistent UI state may include:
• workspace layout
• widget visibility
• widget position
• widget size

• table columns
• sorting
• filters
• selected timeframe

Business data shall not be stored as UI layout state.

UI state and business persistence shall remain separate.

UI Review Checklist

Before introducing or changing UI functionality verify:

• trading workflow identified
• user context preserved
• active context visible
• loading state defined
• stale state defined where relevant
• unavailable state defined
• disconnected state defined where relevant
• error state defined
• keyboard interaction considered
• information density appropriate
• financial values formatted consistently
• critical actions protected
• UI thread remains responsive
• Widget Catalog synchronized
• documentation updated

Related Documents

• Product_Vision.md
• Product_Roadmap.md
• Project_Overview.md
• Technical_Specifications.md
• Widget_Catalog.md
• Architecture.md
• Domain_Model.md
• API_Guidelines.md
• Testing_Strategy.md
• AGENTS.md